

Building Perceptron

Developper un classifieur lineaire et expliquer ses limites

Une classe Python avec NumPy

Un notebook unique : tests, EDA et modelisation

Un probleme binaire : diagnostic benin ou malin

Breast Cancer Wisconsin (Diagnostic) | 569 observations | 30 mesures

1. Comment fonctionne notre perceptron ?

Score : $s = w^T x + b$

Prediction : 1 si $s \geq 0$, sinon 0

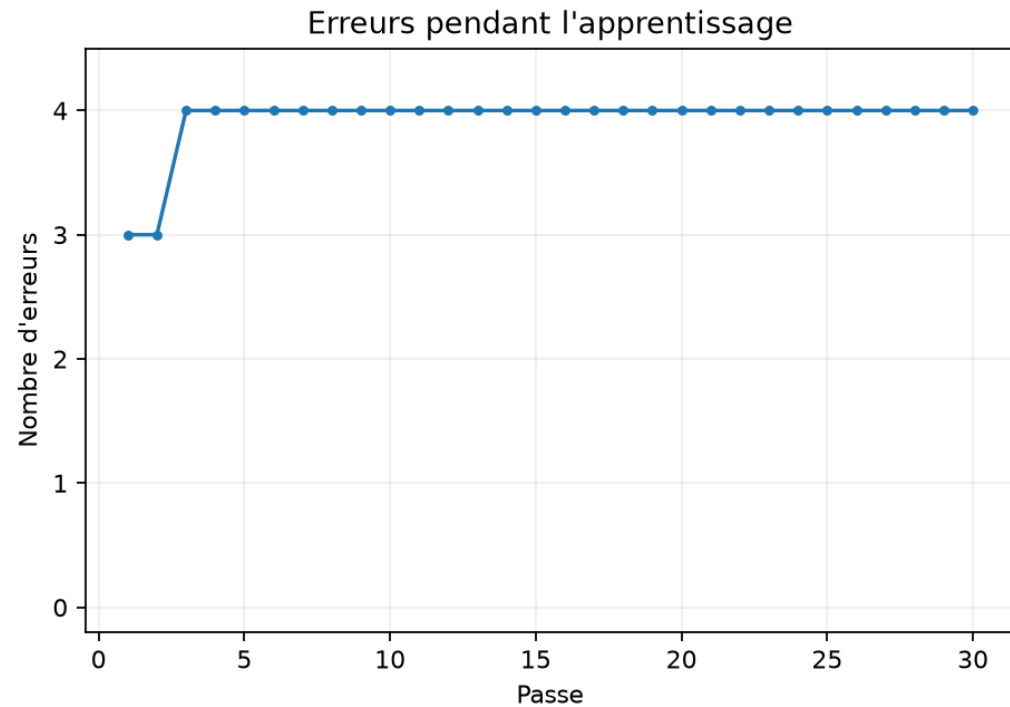
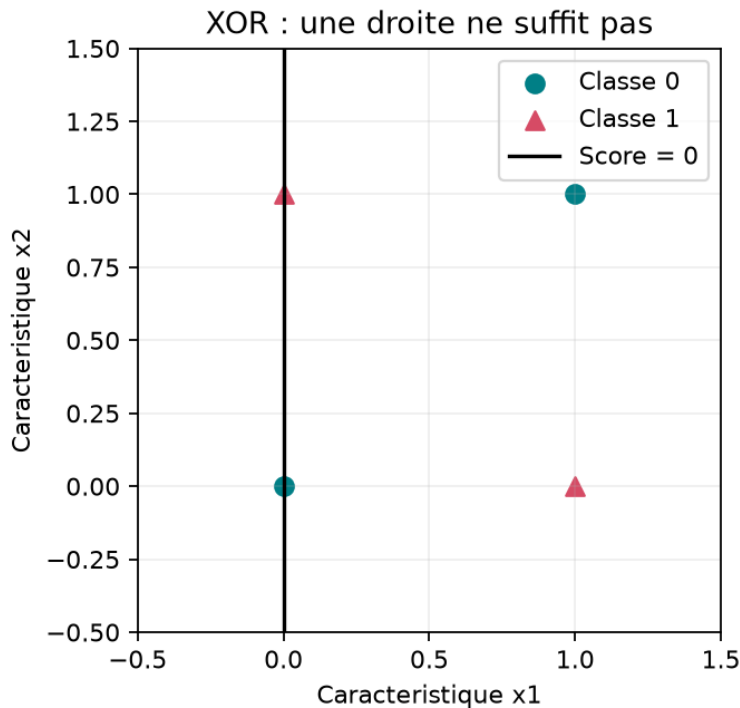
Erreur : $e = y - \text{prediction}$

Correction : $w \leftarrow w + \eta * e * x$; $b \leftarrow b + \eta * e$

fit : apprendre | net_input : calculer le score | predict : classer

errors_ : erreurs pendant chaque passe, pas erreurs du test

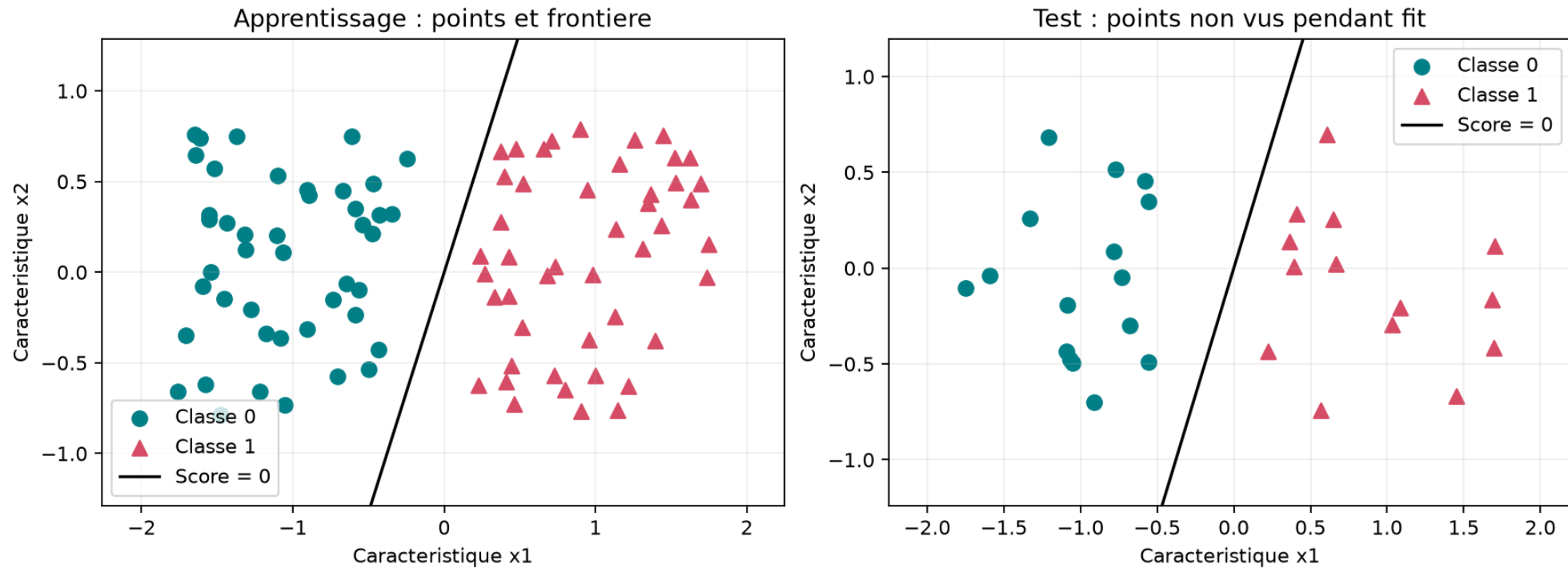
2. Valider la classe : AND reussit, XOR echoue



AND : assertions validees. XOR : 2 erreurs finales sur 4.

L echec attendu sur XOR montre la limite d une frontiere lineaire.

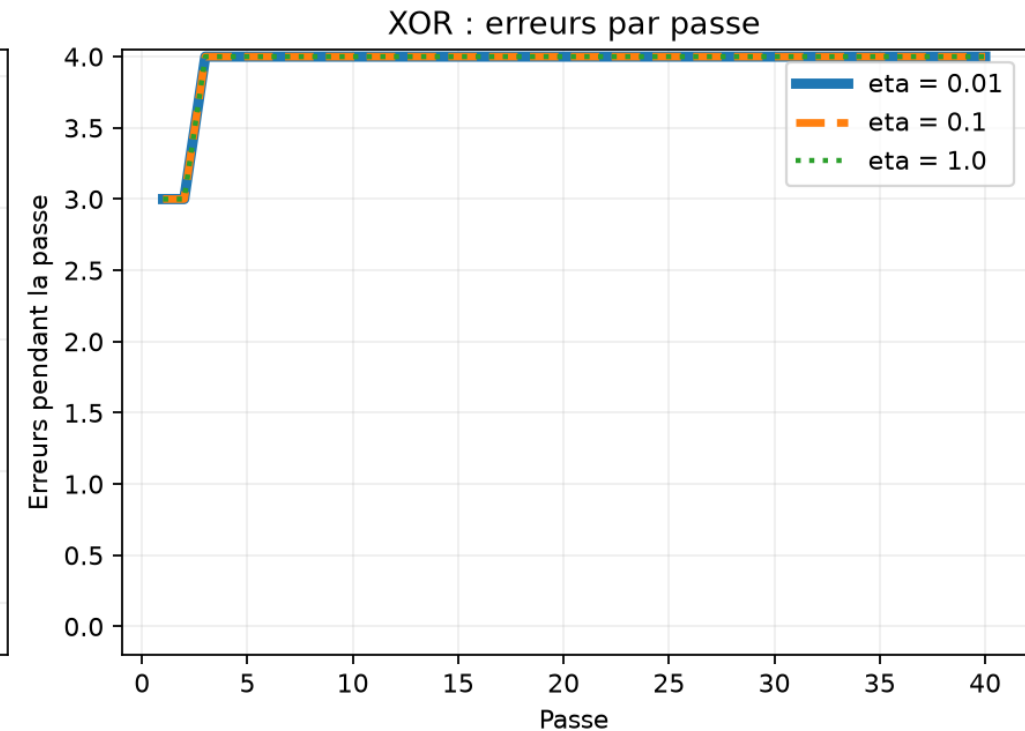
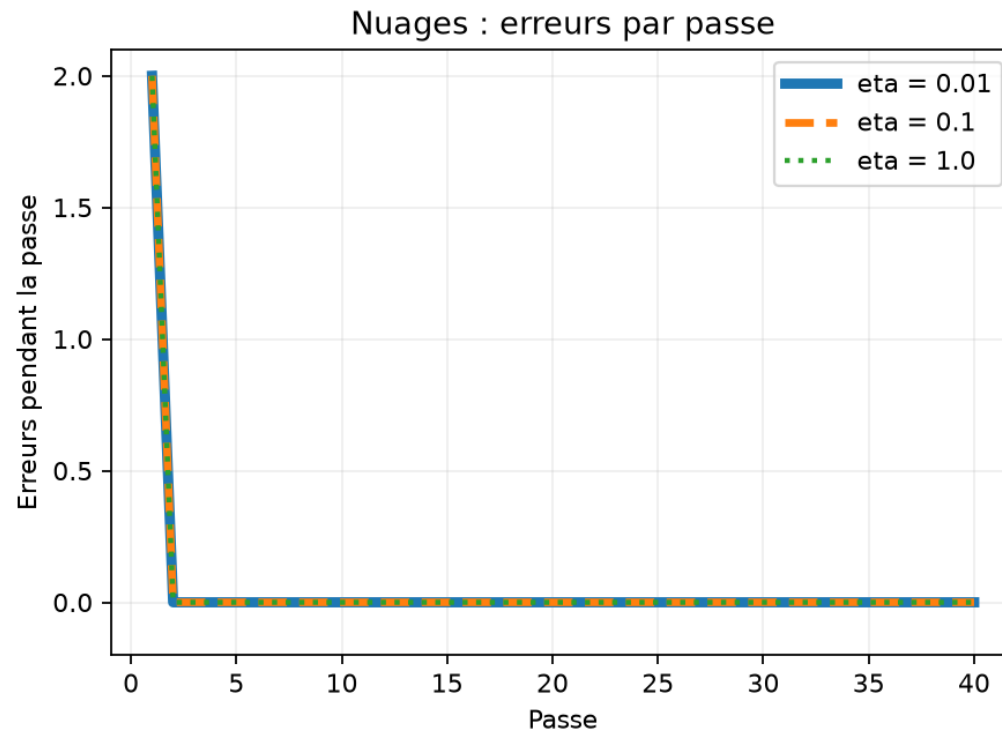
3. Generaliser sur des donnees simulees



90 points pour apprendre | 30 pour tester | accuracy test : 100 %

Un probleme artificiel separable ne prouve pas une fiabilite clinique.

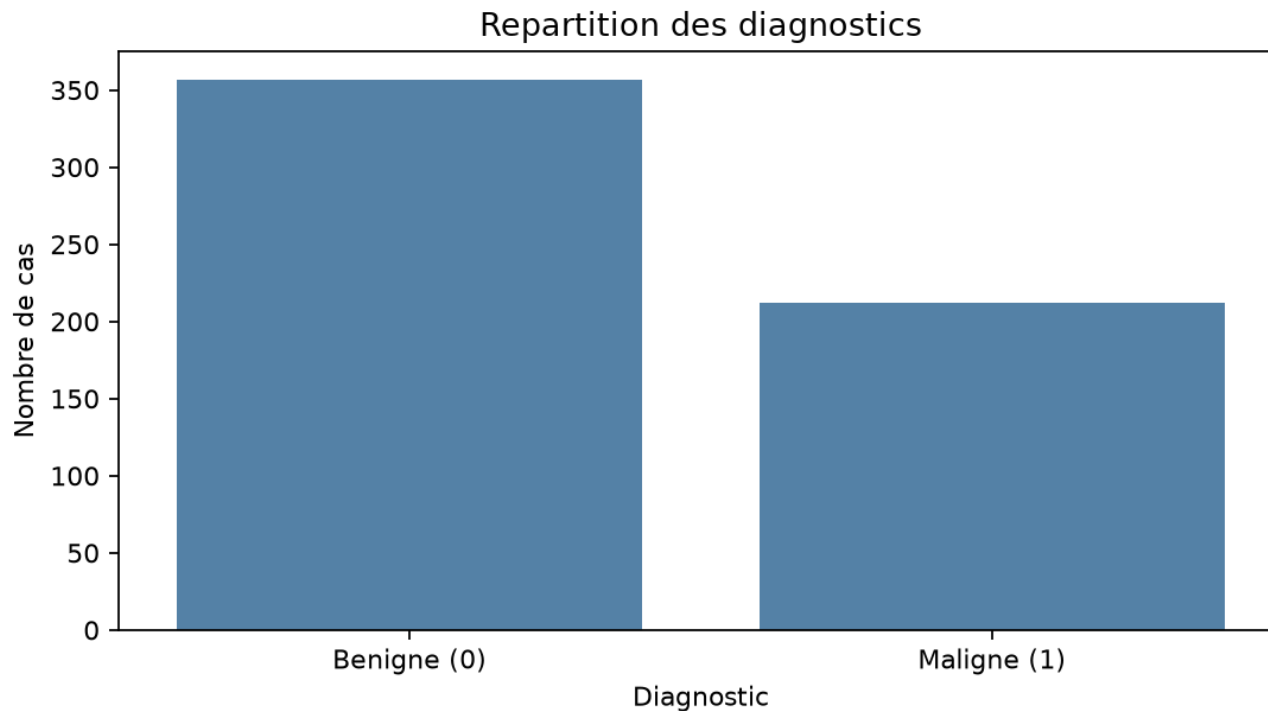
4. Pourquoi les courbes des taux se superposent ?



Nuages : zero erreur. XOR : les corrections persistent.

Ici, changer eta met les poids a une autre echelle sans changer les decisions.

5. Donnees reelles : qualite et cible



569 observations, 30 mesures

357 benignes : 62,7 %

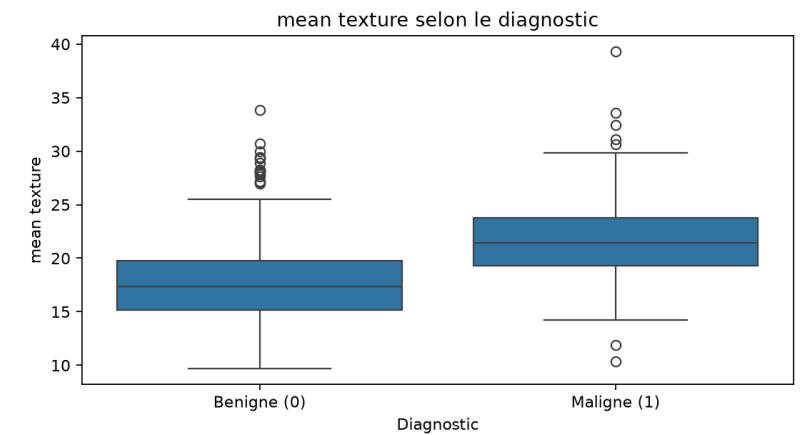
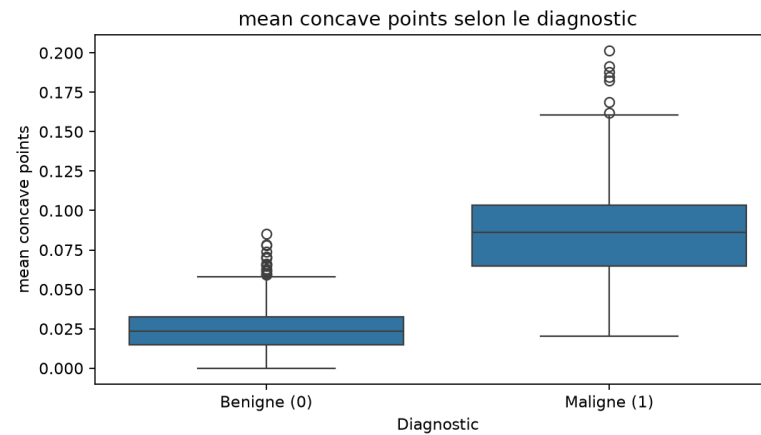
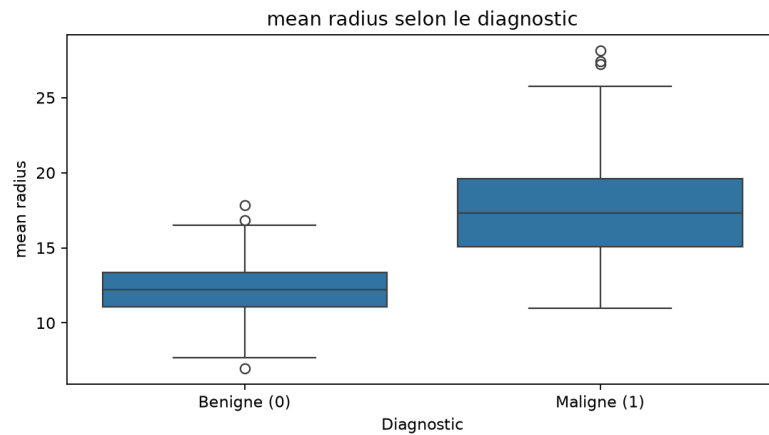
212 malignes : 37,3 %

0 valeur manquante

0 ligne dupliquee

Pas de suppression arbitraire

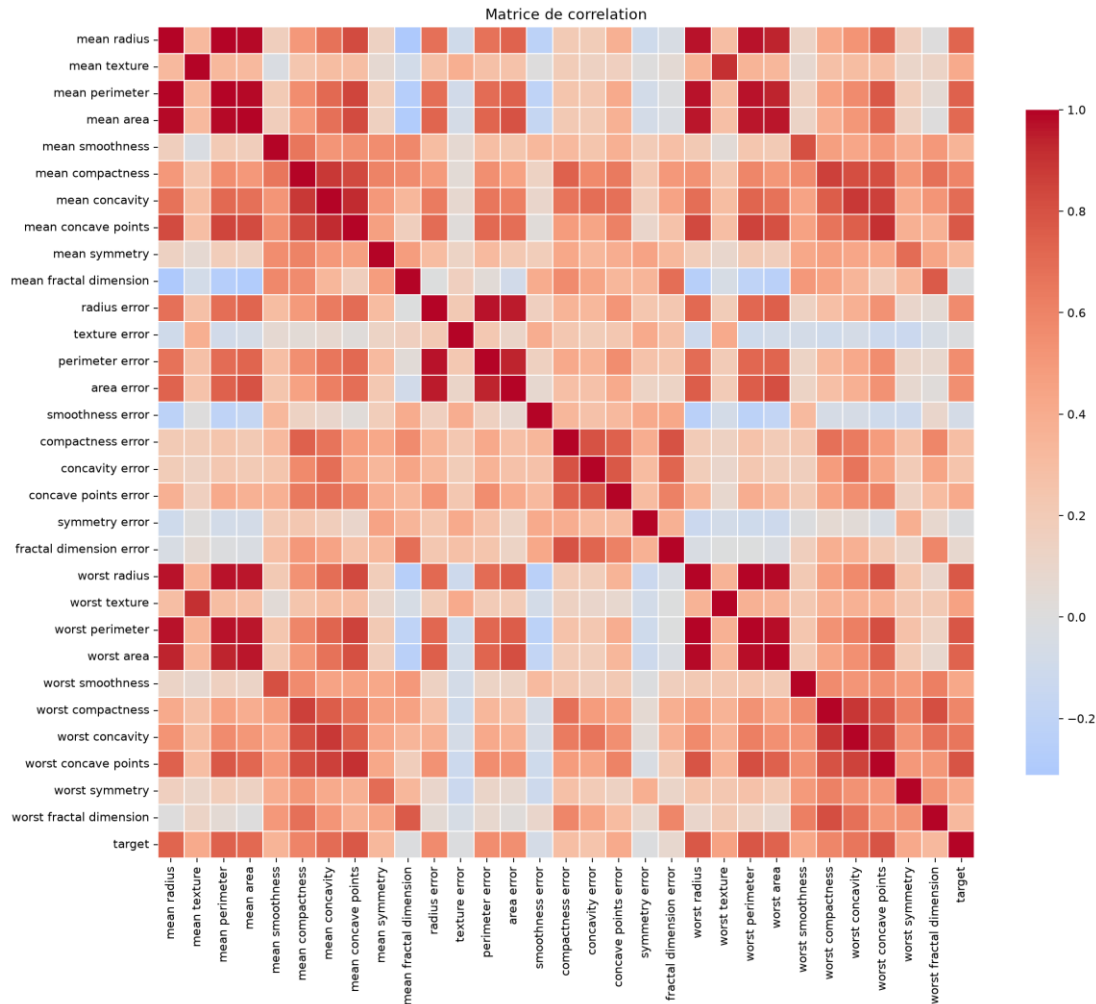
6. EDA : quelles distributions distinguent les classes ?



Rayon et points concaves : separation plus nette que la texture.

Des chevauchements subsistent. Les 30 mesures restent utilisees.

7. EDA : echelles et correlations



Rayon moyen : environ 7 a 28

Aire moyenne : 143,5 a 2501

Autres mesures : proches de 0,1

Echelles differentes : standardiser.

Mesures liees : tester l ACP.

Correlation ne signifie pas causalite.

8. Preparer sans donner les reponses au modele

455 exemples pour apprendre | 114 exemples pour tester

X : 30 mesures, sans target | y : vrai diagnostic

Separation stratifiee : proportions conservees

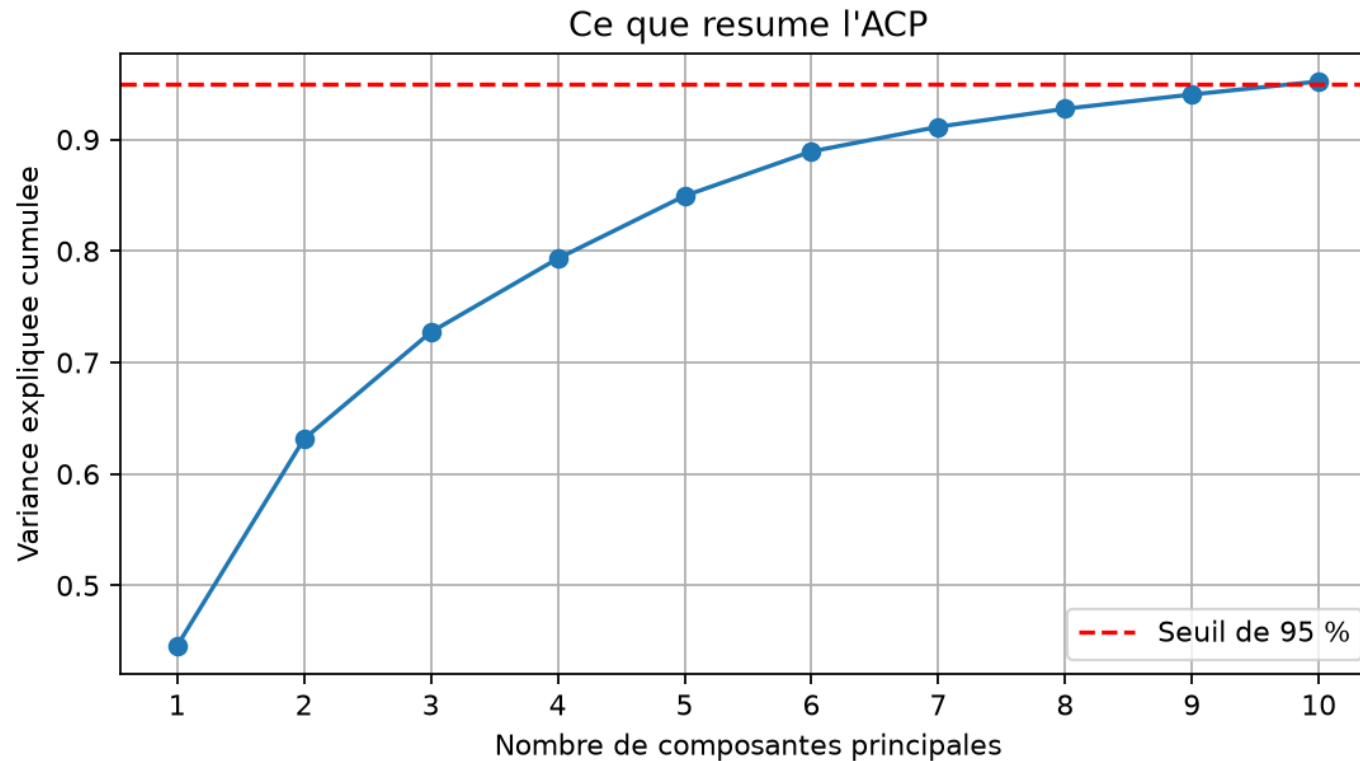
Graine 42 : tirage reproductible

StandardScaler.fit et PCA.fit : apprentissage seulement

transform : meme regle appliquee au test

La cible reste codee 0/1.

9. ACP : compresser les mesures, pas choisir les reponses

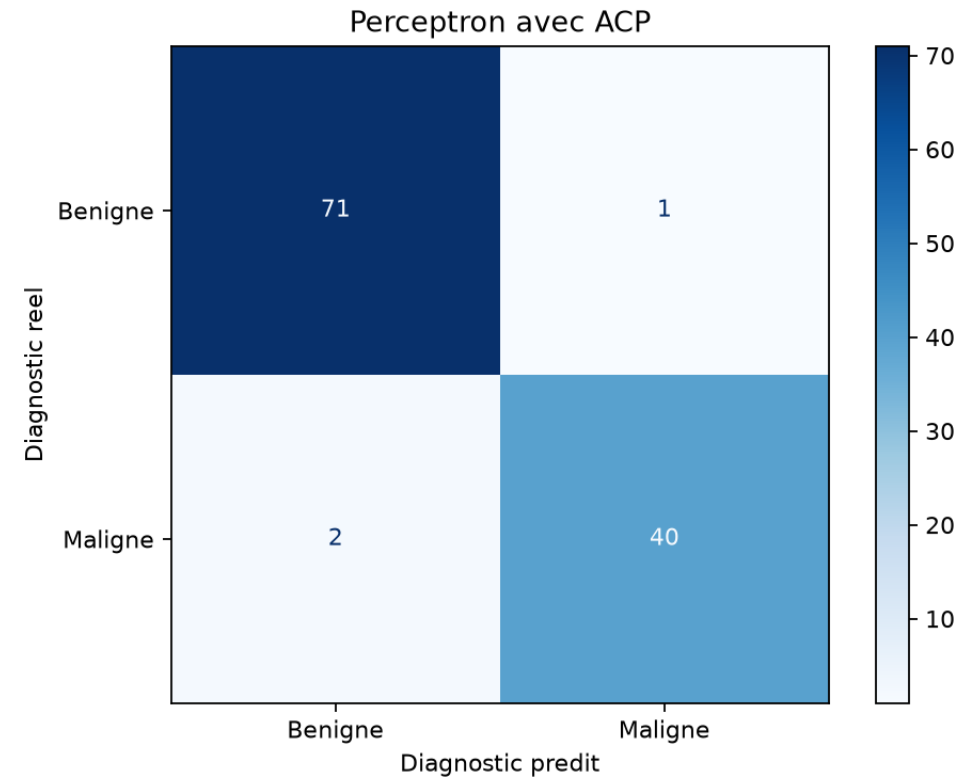
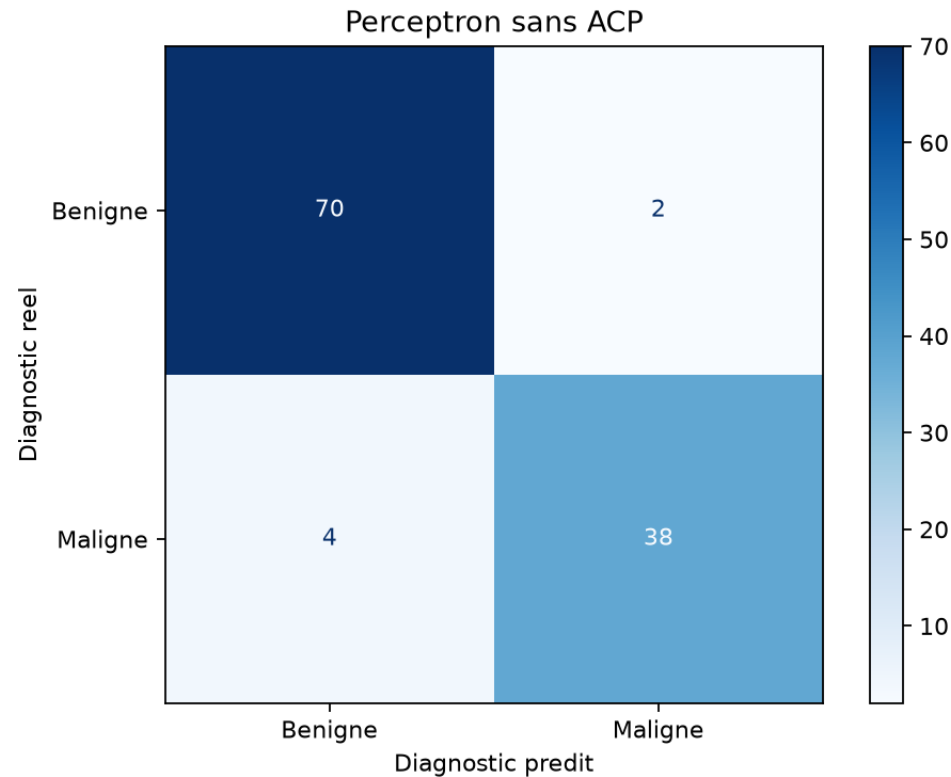


30 mesures -> 10 composantes
95,2 % de variance conservee

Ce n est pas 95,2 % de bonnes
predictions.

On compare avec et sans ACP
sur le meme test.

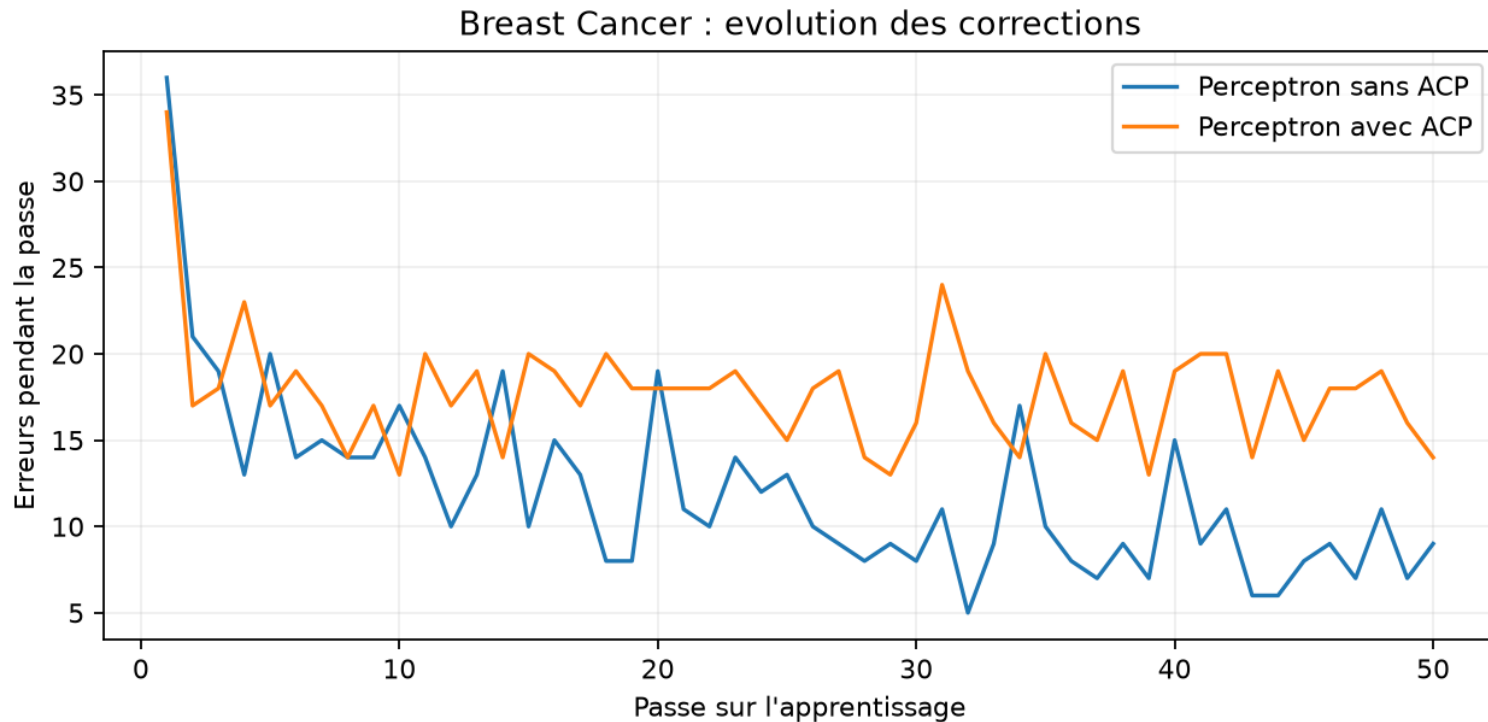
10. Lire les matrices de confusion



Lignes = realite | Colonnes = prediction | Diagonale = bonnes reponses

En bas a gauche : cas malins manques. Sans ACP : 4; avec ACP : 2.

11. Apprentissage et test ne mesurent pas la meme chose



50 passes, $\eta = 0,01$

Les corrections baissent globalement, mais restent non nulles.

Le test est evalue separement avec accuracy, rappel et matrices.

12. Comparaison sur les memes 114 exemples

Modele	Accuracy	Rappel malin	Malins rates	Fausse alertes
Perceptron sans ACP	94.7 %	90.5 %	4	2
Perceptron avec ACP	97.4 %	95.2 %	2	1
Regression logistique sans ACP	96.5 %	92.9 %	3	1
Regression logistique avec ACP	97.4 %	92.9 %	3	0

Meme accuracy ne signifie pas memes erreurs.

Les faux negatifs malins et les fausses alertes doivent etre distingues.

13. Conclusion, limites et perspectives

Classe developpee, tests fonctionnels et limites expliquees

Donnees controlees : pas de nettoyage artificiel

ACP : moins de dimensions et meilleur perceptron sur ce test

Limites : petit jeu de reference, un seul tirage, EDA sur tout le jeu.

Les parametres testes ne sont pas prouves optimaux.

Suite : validation croisee stratifiee avec preprocessing dans chaque pli,
puis test final non consulte pendant le developpement.

Aucune validation clinique.

14. Sources et livrables

Rosenblatt (1958), The Perceptron : doi.org/10.1037/h0042519

UCI, Breast Cancer Wisconsin (Diagnostic) : doi.org/10.24432/C5DW2B

scikit-learn : donnees, StandardScaler, PCA, evaluation, LogisticRegression

References completes et installation dans README.md

[perceptron.py](#) : classe NumPy

[eda.ipynb](#) : notebook unique, propre et interprete

[README.md](#) : contexte, analyse, outils, conclusion, bibliographie

[Slide/](#) : presentation PPTX et PDF | [doc/](#) : reponses en PDF